Lumen — Portfolio Documentation

AI-Based Web Accessibility Checker

Repository: https://github.com/balisikh/ai-based-web-accessibility-checker

Local app: http://localhost:4376

Document date: **2026-08-04** (includes scan PDF export + issue detail UX)

> Assistive findings only — this tool is **not** a legal accessibility certificate or formal conformance audit.

Overview

Lumen scans a public URL for WCAG-oriented accessibility issues, returns a clear score, severity breakdown, and links to rule help. Optional AI tips suggest fixes for top issues. A separate **batch dashboard** shows Pass/Fail results for 28 well-known public websites.

What it does (end-to-end)

1. Paste a public website URL on the home page
2. Lumen opens the page in a headless browser (Chrome via Playwright)
3. **axe-core** checks WCAG A/AA-oriented rules
4. You get a **score**, severity breakdown, and issue details
5. Optionally, **AI tips** explain top issues and suggest fixes
6. Export a **JSON** or **PDF** report

Technology stack

Core application stack (summary):

Layer	Technology
Frontend / API	Next.js 16 (React) + TypeScript
Scanner	Playwright (Chrome) + axe-core
Data	Postgres or local PGlite
AI (optional)	OpenAI-compatible Chat Completions API
CI/CD	GitHub Actions, Docker, GHCR

Full technology reference (everything used to build, style, test, and deploy Lumen):

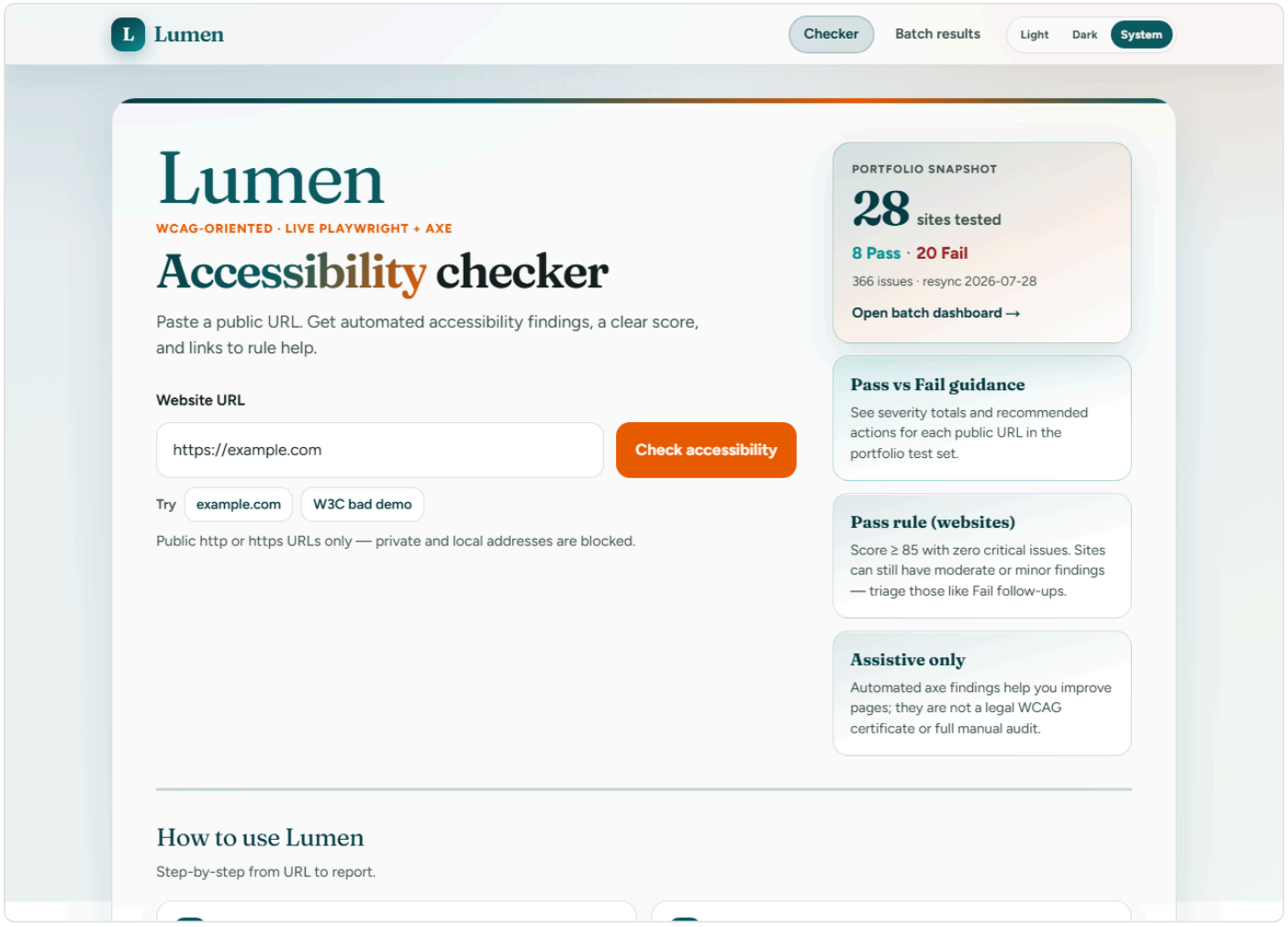
Category	Technology	Role in Lumen
Language	**TypeScript**	Primary language — app (`src/`), API routes, scripts, types
Language	**JavaScript**	Runtime (Node.js); React/Next compiles TS to JS for browser and server
UI framework	**React 19**	Components, client interactivity (`ScanExperience` , theme toggle, batch refresh)
App framework	**Next.js 16**	App Router, SSR, API routes (`/api/scans` , `/api/batch/*`), SEO routes
Styling	**CSS**	`globals.css` — layout, themes, responsive breakpoints, Pass/Fail colours
Styling	**Tailwind CSS v4**	Utility classes + PostCSS pipeline (`@tailwindcss/postcss`)
Styling	**CSS custom properties**	Light / Dark / System themes via `[data-theme]` variables
Scanner	**Playwright**	Headless Chrome — page render, batch rescans, viewport CI, PDF export
Scanner	**axe-core** + **@axe-core/playwright**	WCAG A/AA rule engine on rendered DOM
Database	**PGlite** (`@electric-sql/pglite`)	Embedded Postgres for local dev — `web/data/lumen-pg`
Database	**PostgreSQL** + `pg` driver	Production persistence via `DATABASE_URL`
AI	**OpenAI-compatible REST API**	Optional Chat Completions for issue tips (`ai-enrichment.ts`)
Runtime	**Node.js 22**	Dev server, build, scripts, CI (GitHub Actions)
Build	**Webpack**	Stable CSS/JS bundling (`--webpack` on dev/build — avoids Turbopack issues on low RAM)
Package manager	**npm**	Dependencies, scripts (`package.json` , `package-lock.json`)
Script runner	**tsx**	Run TypeScript maintenance scripts without pre-compile
Lint / quality	**ESLint** + **eslint-config-next**	Code style and Next.js rules — part of `npm run ci`
Images	**sharp**	PDF screenshot crops (`prepare-pdf-screenshots.ts`)
Version control	**Git**	Source history, branches, commits; required for CI/CD and batch snapshot sync
Remote repo	**GitHub**	Hosting, pull requests, Actions workflows, GHCR container registry
CI	**GitHub Actions**	`ci.yml` (lint, typecheck, validate, build, viewport); `batch-rescan.yml` ; `cd.yml`
Containers	**Docker** + **Dockerfile**	Multi-stage image — deps → build → runner with Chromium (`web/Dockerfile`)
Registry	**GHCR**	`ghcr.io/<owner>/lumen-web` — production image from CD workflow
Local automation	**PowerShell** (`.ps1`)	Windows helpers — `start-local.ps1` , `fix-styles.ps1` (rebuild when CSS breaks)

Browser (local scan)	**Google Chrome**	Playwright `channel: "chrome"` for live scans on developer machines
Browser (CI/Docker)	**Chromium**	`PLAYWRIGHT_CHROMIUM=1` / `playwright install chromium` in CI and container

Note: TypeScript and JavaScript are both part of the stack — you **write** TypeScript; **Node and the browser run** JavaScript after compile. **CSS** (not a separate CSS framework file per component) drives all visual design in `globals.css` with Tailwind v4 integration. **Git** is essential to the workflow: every push triggers CI; weekly batch rescan **commits** updated snapshot data to `main`.

UI screenshots (styled layout)

Home — accessibility checker (hero)



Home — complete page (how-to, rules & FAQ)

How to use Lumen

Step-by-step from URL to report.

1 Enter a public URL
Paste an http or https link in the field above (for example https://example.com), or pick a Try example.

2 Check accessibility
Click Check accessibility to scan that single page. Localhost and private network addresses are blocked.

3 Wait for progress
Follow the status steps while Lumen fetches the page, runs rules, optional AI tips (when enabled), and calculates a score. Most public pages finish within about a minute.

4 Review findings
Check the score and severity counts. Select an issue to see WCAG mapping, HTML snippet, rule help, and AI guidance when enabled on this server. Filter by severity if the list is long.

5 Export JSON (optional)
On the results screen, use Export JSON to download the report for tickets, docs, or automation.

How Lumen checks a page

AI tips off

1 Public websites
Use http or https links that anyone can open on the internet.
✓ https://example.com ✗ localhost or 127.0.0.1 ✗ 192.168.x.x

2 Real browser scan
Automated Chrome loads the page in the background, like a real visitor.

3 Accessibility rules
Industry-standard checks (axe-core) for common WCAG Level A and AA requirements.

4 Optional AI tips
Not enabled on this Lumen server — you still get rule findings, WCAG mapping, and a score.

Website batch results

+ What URLs can I use?

+ What are the limits of a scan?

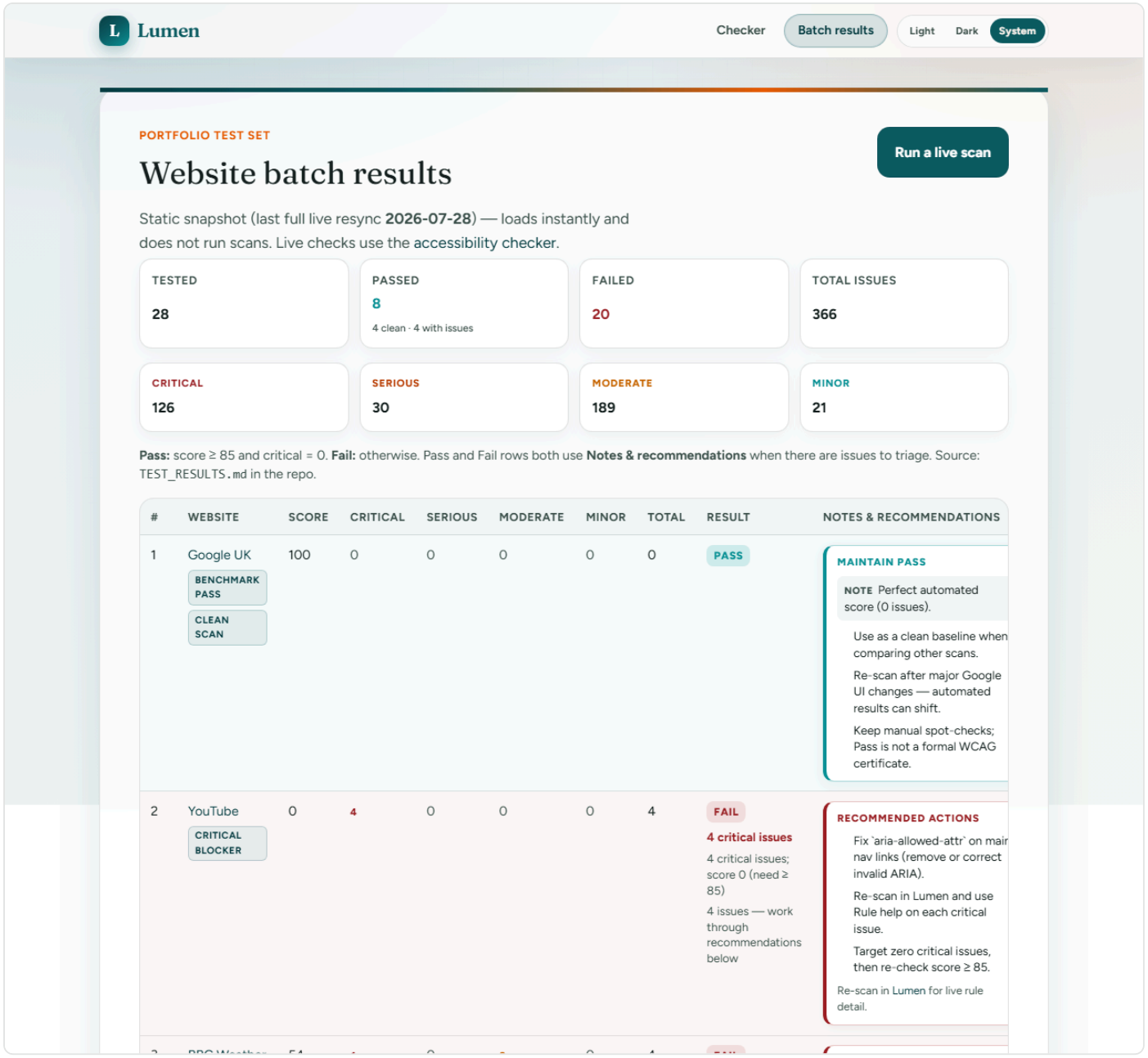
+ How is the score calculated? What does Pass / Fail mean?

+ What are AI tips? Why are they off?

+ Why did my scan fail or say rate limited?

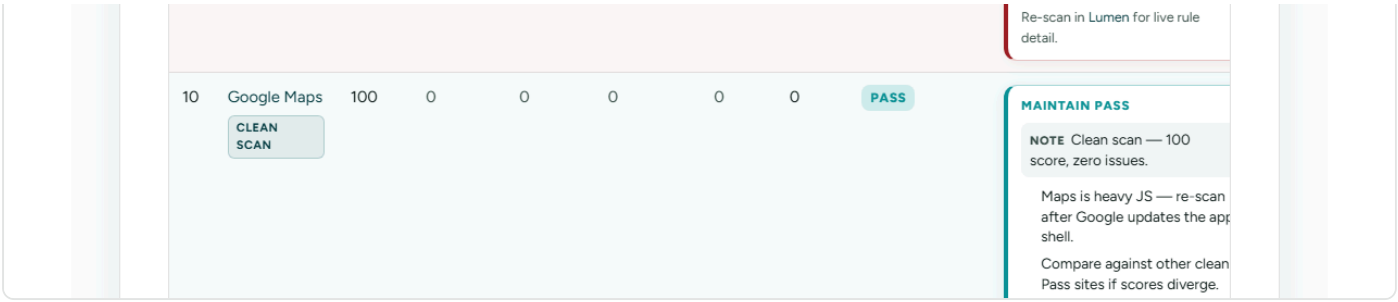
+ What happens to my URL and scan data?

Batch results — 28 sites (overview)

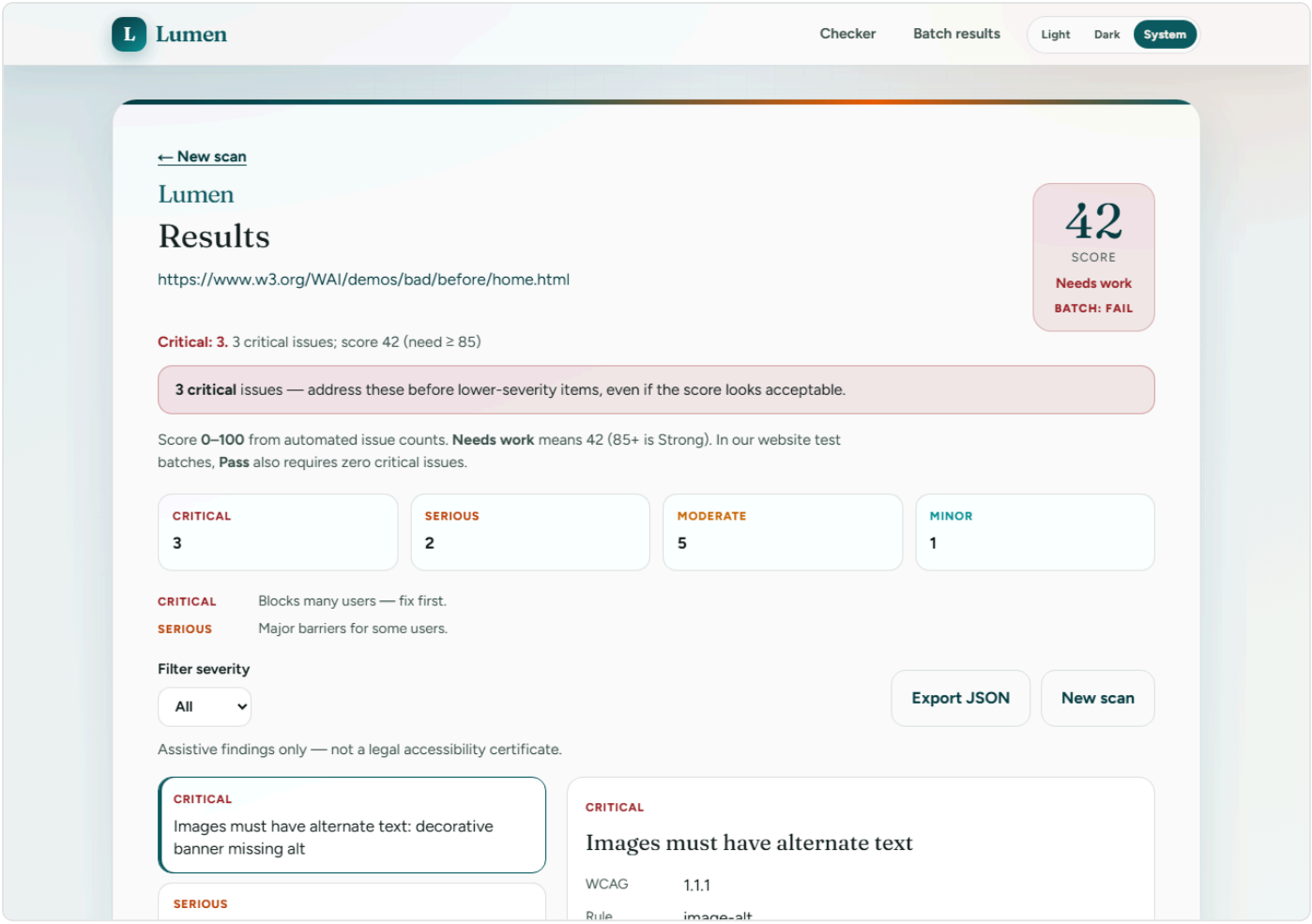


Batch results — complete page (website table)

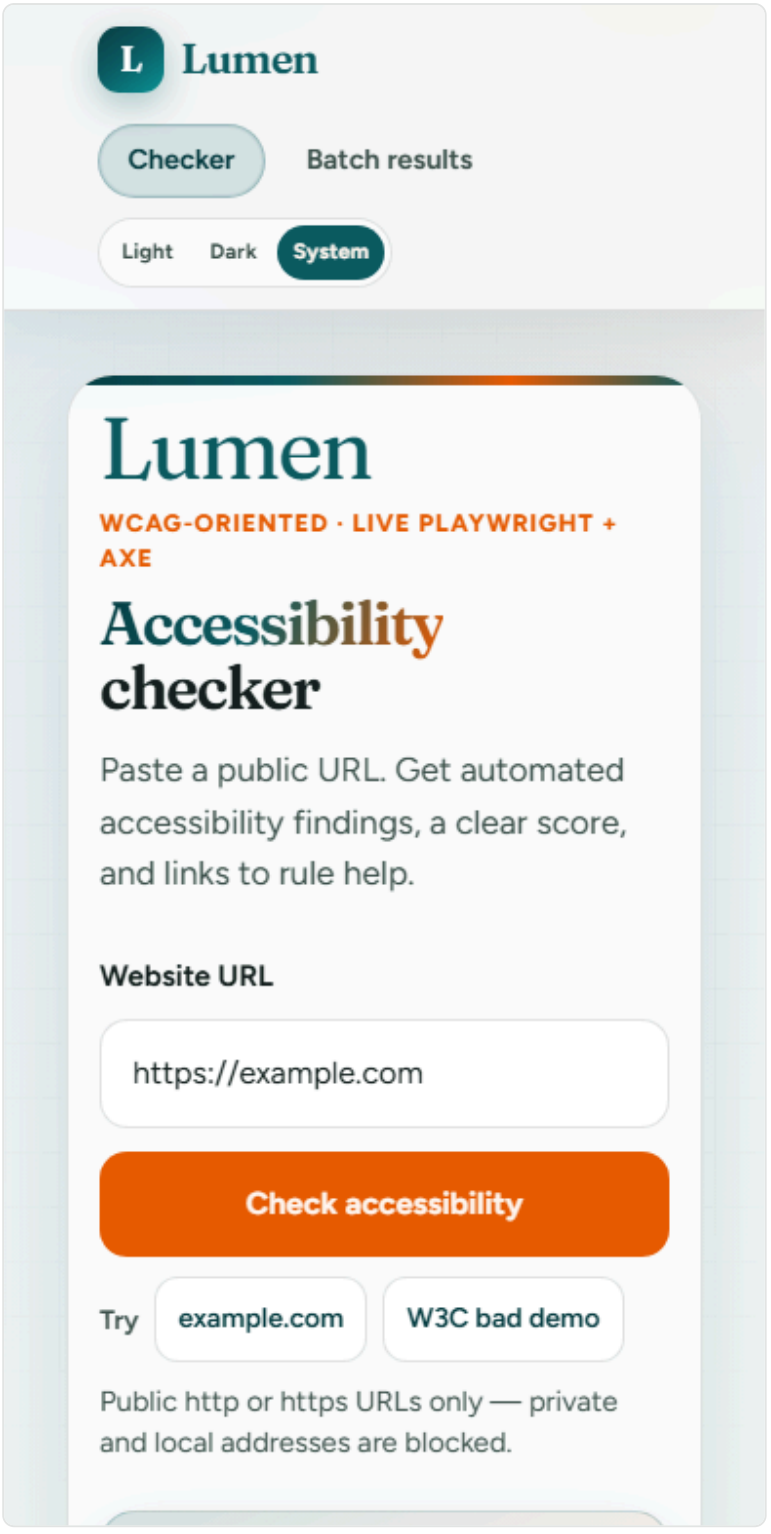
3	BBC Weather Southall CRITICAL BLOCKER	54	1	0	3	0	4	FAIL 1 critical issue 1 critical issue; score 54 (need ≥ 85) 4 issues — work through recommendations below	RECOMMENDED ACTIONS Fix critical `aria-required-attr` first. Resolve `aria-hidden-focus` and `color-contrast` (serious) on weather UI. Re-scan after BBC deploys changes (page content changes often). Re-scan in Lumen for live rule detail.
4	BBC iPlayer PASS WITH ISSUES	86	0	0	2	0	2	PASS 2 issues — batch Pass; triage below (keep critical at 0)	RECOMMENDED ACTIONS NOTE Pass with 2 moderate issues (score 86). Open each moderate issue in Lumen Rule help and fix when convenient. Re-scan to confirm critical stays at 0 and score stays ≥ 85. Prioritize any new critical findings before release. Still a batch **Pass** — re-scan in Lumen after fixes so critical stays 0
5	BBC News PASS WITH ISSUES	93	0	0	1	0	1	PASS 1 issue — batch Pass; triage below (keep critical at 0)	RECOMMENDED ACTIONS NOTE Pass with 1 moderate issue (score 93). Triage the moderate finding; low effort may lift score further. Re-scan after BBC template or component updates. Export JSON if tracking regressions in a ticket. Still a batch **Pass** — re-scan in Lumen after fixes so critical stays 0
6	Disney+ UK CLEAN SCAN	100	0	0	0	0	0	PASS	MAINTAIN PASS NOTE Clean scan — 100 score, zero issues. Re-scan periodically; streaming sites update often. Use as a Pass reference alongside Google UK / Maps. Still validate key flows manually (keyboard, captions, etc.).
7	GitHub balisikh CLEAN SCAN	100	0	0	0	0	0	PASS	MAINTAIN PASS NOTE Profile page Re-scan if GitHub changes profile markup or ARIA patterns. Scan other URLs separately if testing org/repo pages. Maintain zero critical on re-scan to keep batch Pass.
8	ChatGPT CRITICAL BLOCKER	50	1	1	1	1	4	FAIL 1 critical issue 1 critical issue; score 50 (need ≥ 85) 4 issues — work through recommendations below	RECOMMENDED ACTIONS Correct unsupported ARIA (`aria-allowed-attr`, critical). Improve text contrast (`color-contrast`, serious). Re-scan; export JSON to track before/after. Re-scan in Lumen for live rule detail.
9	Spotify Web Player RE-VERIFY SCAN CRITICAL BLOCKER	0	82	0	2	16	100	FAIL 82 critical issues 82 critical issues; score 0 (need ≥ 85) 100 issues — work through recommendations below	RECOMMENDED ACTIONS NOTE Re-scan: app shell / login — confirm in UI Confirm URL and cookies (open.spotify.com vs marketing landing). Triage 82 critical findings — fix invalid ARIA/roles first. Re-scan after stable page load; compare to prior Pass (100/0).



Results — sample layout



Mobile home (390px)



Batch snapshot (2026-08-04)

Static dashboard at `/batch` — loads instantly from committed or live JSON data.

Metric	Value
Websites tested	**28**
Passed	**8** (4 clean · 4 with issues)
Failed	**20**
Total issues	**330**
Last live refresh	**28/28 OK** (~88 seconds, parallel background job)

Pass / Fail rule

Result	Rule
Pass	Score **≥ 85** and Critical issues **= 0**
Fail	Score **< 85** or Critical issues **≥ 1**

Severity totals (all 28 scans)

Critical	Serious	Moderate	Minor
46	37	217	30

Feature reference

Each feature below follows the same structure: **what it is**, **why we use it**, **key purpose**, **how we implemented it**, and **how to use it**.

1. Live URL scanning (Playwright + axe-core)

What it is	**A real browser scan of one public URL. Playwright launches headless Chrome, loads the page, and axe-core runs WCAG 2.x A/AA tagged rules.**
Why we use it	Static HTML checks miss client-rendered content. axe-core is industry-standard for automated accessibility testing and maps to WCAG success criteria.
Key purpose	Give developers a fast, trustworthy signal on real page behaviour — not just source code.
How we implemented it	`web/src/lib/live-scan.ts` — shared browser instance, navigation fallbacks (`domcontentloaded` → `load` → `commit`), axe tags `wcag2a` , `wcag2aa` , `wcag21a` , `wcag21aa` , `wcag22aa` . Violations mapped in `axe-mapper.ts` . Pipeline wired from `POST /api/scans` .
How to use it	Home page → paste URL → **Check accessibility**. Progress shows fetching → rendering → rule analysis. Results appear when complete.

2. Scan workflow UI (home → progress → results)

What it is	The main user journey: URL input, live progress, score panel, issue list, and detail view.
Why we use it	Scanning can take 10–60 seconds. Clear states reduce anxiety and match the "fast first report" product goal.
Key purpose	Make accessibility checking feel simple for non-experts while still showing enough detail for developers.
How it is implemented	`ScanExperience.tsx` (client) polls `GET /api/scans/:id`. Server components pass batch snapshot context. Copy from `how-to-use.ts` and `product-copy.ts`. Issue detail panel: theme-aware surfaces, readable snippet/selector boxes, severity pills, context notes for `color-contrast` and landmark rules (`globals.css`). Fixture route `/fixtures/results` for layout QA without live scans.
How to use it	Open `/`. Enter a URL or click a **Try example** chip. Watch progress. Click an issue for rule help and (if enabled) AI tips. **Export JSON** or **Export PDF** when done.

3. Scoring and Pass / Fail rules

What it is	A 0–100 score derived from issue severities, plus a binary Pass/Fail gate for the batch dashboard.
Why we use it	Teams need one number for triage and a clear portfolio benchmark across 28 sites.
Key purpose	Deterministic scoring (rules find issues; score is formula-based, not AI-generated).
How we implemented it	`store.ts` — `scoreFromIssues()`, `countBySeverity()`. Pass rule in `website-pass-fail.ts`: Pass if score ≥ 85 **and** critical = 0. Documented in `product-copy.ts` and home FAQs.
How to use it	Results panel shows score + severity chips. Batch table shows Pass/Fail per site. After scan → **Export JSON** or **Export PDF**.

4. URL validation and SSRF protection

What it is	Server-side checks that block scanning of private networks, localhost, and non-public hosts.
Why we use it	Without this, a scanner could be abused to probe internal IPs (SSRF).
Key purpose	Security by default — only public web pages are allowed.
How we implemented it	`ssrf.ts` — hostname blocklist, DNS resolve, reject private/reserved IP ranges. Called from `live-scan.ts` before navigation. User-facing errors in FAQs (`home-faqs.ts`).
How to use it	Use normal `https://` public URLs. Localhost and `192.168.x.x` are rejected with a clear message.

5. Rate limiting (live scans and batch refresh)

What it is	**Per-IP limits on how often clients can start expensive operations.**
Why we use it	Playwright scans are CPU/RAM heavy. Limits prevent abuse and keep the app responsive for everyone.
Key purpose	Protect the server without requiring user accounts.
How we implemented it	`rate-limit.ts` — in-memory sliding window. `POST /api/scans`: default 5/min (production). Batch refresh: `batch-refresh-config.ts` — 5 per 10 min locally, 1/hour in production CI hosts. Returns **429** + `Retry-After`.
How to use it	If you hit the limit, wait for the retry time shown. Override via `RATE_LIMIT_MAX`, `BATCH_REFRESH_RATE_LIMIT_*` in `.env.local` for self-hosted setups.

6. Persistence (PGlite and Postgres)

What it is	**Scan records and issues stored in a database so results survive page reloads and server restarts.**
Key purpose	Shareable scan URLs, JSON export, and audit trail without login.
Why we use it	In-memory-only storage would lose data on every deploy; Postgres is needed for production.
How we implemented it	`db/` schema + async scan store. Default: **PGlite** at `web/data/lumen-pg`. Production: set `DATABASE_URL` for Postgres. Health check at `GET /api/health`.
How to use it	No setup locally — PGlite creates files automatically. For production, set `DATABASE_URL` in host env (see `web/DEPLOY.md`).

7. JSON export

What it is	**Download of the full scan report as structured JSON.**
Why we use it	Developers and QA tools need machine-readable output for tickets, CI, or archival.
Key purpose	Machine-readable handoff for developers and automation.
How we implemented it	`GET /api/scans/:id/export?format=json` — issues, score, metadata. **Export JSON** button on results panel.
How to use it	After a scan completes → **Export JSON** on the results view.

7b. Scan report PDF export

What it is	**Downloadable formatted PDF report after a scan completes — summary, score, Pass/Fail, severities, and full issue list.**
Why we use it	Stakeholders, tutors, and QA often need a readable document, not raw JSON.
Key purpose	Shareable report for tickets, portfolios, and hand-ins.
How we implemented it	`GET /api/scans/:id/export?format=pdf` — `scan-report-pdf.ts` builds printable HTML, Playwright renders A4 PDF. **Export PDF** button on results (`ScanExperience.tsx`). Returns **409** until scan completes.
How to use it	Complete a scan → **Export PDF** on results, or open `/api/scans/{id}/export?format=pdf`.

8. Optional AI tips

What it is	**LLM-generated explanation and suggested fix for the top N issues by severity.**
Why we use it	axe tells you *what* failed; AI helps explain *why it matters* and *how to fix it* for developers who are new to a11y.
Key purpose	Hybrid detection: rules find issues; AI enriches — scoring stays deterministic.
How we implemented it	`ai-enrichment.ts` — OpenAI-compatible API. Env: `AI_API_KEY` / `OPENAI_API_KEY`, `AI_MODEL`, `AI_MAX_ISSUES`. Failures never block the report. `GET /api/config` exposes `aiTipsEnabled` without secrets.
How to use it	Add key to `web/.env.local`, restart server. Top issues show **AI tip** sections when enabled.

9. Batch results dashboard (28 sites)

What it is	**A pre-built Pass/Fail table for 28 public websites (Google, BBC, Netflix, W3Schools, etc.).**
Why we use it	Demonstrates Lumen on real-world pages instantly — no waiting for 28 live scans on every visit.
Key purpose	Portfolio proof and regression benchmark across diverse site types (SPAs, media, e-commerce, docs).
How we implemented it	`website-batch-results.ts` — committed snapshot. `/batch` page renders summary tiles + sortable table. Fail guidance in `website-batch-fail-guidance.ts`. Unified date via `getBatchSnapshot()` in `batch-snapshot-store.ts`.
How to use it	Nav → **Batch results**. Review Pass/Fail, scores, severities. Click **Run a live scan** to test your own URL.

10. Refresh snapshot (background batch rescan)

What it is	**A button on `/batch` that live-rescans all 28 sites and saves a runtime JSON snapshot.**
Why we use it	Keeps the dashboard up to date without a manual CLI run or waiting for weekly CI.
Key purpose	Fast refresh (~under 2 minutes) without blocking the rest of the app.
How we implemented it	`POST /api/batch/refresh` → **202 Accepted**, background job in `batch-refresh-job.ts`. Parallel rescans (`BATCH_RESCAN_CONCURRENCY=2`) in `batch-live-rescan.ts`. Per-site 90s timeout. Progress polling via `GET /api/batch/refresh`. UI: `BatchRefreshButton.tsx` shows `Scanning SiteName (N/28)…`. Saves `web/data/batch-live-snapshot.json`. Enabled in dev or with `BATCH_REFRESH_ENABLED=1`.
How to use it	`/batch` → **Refresh snapshot**. Watch progress. Page reloads when done. Tune speed via `.env.local` (see `web/README.md`).

11. Batch sync CLI and weekly CI rescan

What it is	**Automated pipeline to rescan all 28 sites and commit updated scores to git.**
Why we use it	Deployed sites read committed TypeScript data — runtime JSON alone does not survive fresh deploys.
Key purpose	Keep production and git in sync with real-world site changes over time.
How we implemented it	`npm run batch:sync` = `batch:rescan` → `batch:apply-rescan` → `validate:batch`. GitHub Actions: `.github/workflows/batch-rescan.yml` — Sunday 04:00 UTC + manual dispatch; commits `website-batch-results.ts` when changed.
How to use it	Local: `cd web && npm run batch:sync`, then commit. CI: Actions → **Batch rescan** → Run workflow.

12. Theme system (Light / Dark / System)

What it is	**User-selectable colour theme with system preference detection.**
Why we use it	Accessibility tools should respect user contrast preferences; dark mode reduces eye strain for long review sessions.
Key purpose	WCAG-aligned product UI — the checker itself should be usable.
How we implemented it	CSS variables in `globals.css` — `[data-theme="light"]`, `dark`, system via `prefers-color-scheme`. Theme toggle in header. Batch page uses readable table/card styles in both themes.
How to use it	Header → **Light**, **Dark**, or **System**. Preference persists in `localStorage`.

13. Responsive layout and viewport CI

What it is	**Layout adapts from mobile (320px) to desktop (1440px+) with device-tier gutters and breakpoints.**
Why we use it	Teams review scans on phones and tablets; overflow bugs break real usage.
Key purpose	One codebase, five layout tiers — mobile cards, tablet batch cards, laptop/desktop tables.
How we implemented it	Breakpoint tokens in `globals.css` (`--bp-mobile` 480, `--bp-tablet` 768, `--bp-laptop` 1024, `--bp-desktop` 1440). `npm run test:responsive` — Playwright checks overflow at 320, 390, 768, 1024, 1280, 1440 on `/`, `/batch`, `/fixtures/results`. CI job **responsive-viewport**.
How to use it	Resize browser or use DevTools device mode. CI runs automatically on push to `main`.

14. SEO (search visibility)

What it is	**Metadata, sitemap, robots.txt, Open Graph, and JSON-LD for public deployment.**
Why we use it	Portfolio and product discoverability when deployed to a public URL.
Key purpose	Correct canonical URLs, social previews, and crawler guidance.
How we implemented it	`seo.ts`, `layout.tsx` metadata, `/sitemap.xml`, `/robots.txt`, `/icon.svg`. Requires `NEXT_PUBLIC_SITE_URL` in production (`web/DEPLOY.md`).
How to use it	Set `NEXT_PUBLIC_SITE_URL` on deploy. Submit sitemap in Google Search Console.

15. Continuous integration (GitHub Actions)

What it is	**Automated checks on every push/PR to `main`.**
Why we use it	Catch lint, type, build, and snapshot regressions before merge.
Key purpose	Confidence that production build and batch data stay valid.
How we implemented it	`.github/workflows/ci.yml` — ESLint, `typecheck:scripts`, `validate:copy`, `validate:batch`, `next build`, responsive viewport job. Badge in root README.
How to use it	`cd web && npm run ci` locally before push. Fix any red checks in Actions tab.

16. Continuous deployment (Docker + GHCR)

What it is	**Docker image build and push after CI passes on `main`.**
Why we use it	Playwright needs a worker-capable host with Chrome — container standardizes deploy.
Key purpose	Repeatable production deploy path.
How we implemented it	`web/Dockerfile`, `.github/workflows/cd.yml` → GHCR `ghcr.io/<owner>/lumen-web`. Optional `RENDER_DEPLOY_HOOK` secret. Documented in `web/DEPLOY.md`.
How to use it	Configure host secrets (`DATABASE_URL`, `NEXT_PUBLIC_SITE_URL`, optional AI key). Pull image or connect deploy hook.

17. Centralized product copy

What it is	**Single source of truth for UI strings, FAQs, Pass/Fail rule, and disclaimers.**
Why we use it	Prevents README and UI drift; FAQs stay consistent across home and docs.
Key purpose	Maintainability — change copy once, validate everywhere.
How we implemented it	`product-copy.ts`, `how-to-use.ts`, `home-faqs.ts`, `try-examples.ts`. `npm run validate:copy` fails CI if README duplicates in-app steps/FAQs.
How to use it	Edit copy in `web/src/lib/`. Run `npm run validate:copy`. Do not duplicate long FAQ text in README.

18. Demo mode (no browser)

What it is	**Pre-canned scan findings without Playwright or network access.**
Why we use it	CI, low-RAM machines, or demos when Chrome is unavailable.
Key purpose	UI development and testing without live scans.
How we implemented it	`USE_DEMO_SCAN=1` env flag — sample issues returned instead of `live-scan.ts`.
How to use it	`$env:USE_DEMO_SCAN="1"; npm run dev` (PowerShell) or `USE_DEMO_SCAN=1 npm run dev`.

19. Local development tooling

What it is	Scripts and defaults for stable development on Windows and low-RAM machines.
Why we use it	Next.js 16 Turbopack could crash compiling large `globals.css` on low memory, leaving unstyled HTML.
Key purpose	Reliable local demos for portfolio and interviews.
How we implemented it	Webpack forced via `--webpack` on `dev` / `build`. `start-local.ps1`, `fix-styles.ps1`, `build:low-mem` / `start:low-mem` (2 GB heap). Troubleshooting in `web/README.md`.
How to use it	If page shows plain white text: `cd web; .\scripts\fix-styles.ps1`. For production-like local: `npm run build:low-mem && npm run start:low-mem`.

MVP feature checklist (summary)

Area	Status
Scan UI (home → progress → results)	Done
Live Playwright + axe-core analysis	Done
URL validation + SSRF protections	Done
Per-IP rate limiting	Done
Persistence (Postgres or PGlite)	Done
JSON export	Done
PDF export (scan report)	Done
Optional AI enrichment	Done
Anonymous use (no login)	Done
Batch dashboard (28 sites)	Done
Refresh snapshot (background)	Done
Weekly CI batch rescan	Done
Responsive layout + viewport CI	Done
SEO (sitemap, OG, JSON-LD)	Done
Docker + CD workflow	Done
Accounts / multi-page crawl	Not yet

MVP features — detailed reference

Each completed MVP item below explains **what it does**, **why it exists**, and **how it is implemented** in Lumen. Deferred items explain what they would add and why they are out of scope for v1.

MVP-1. Scan UI (home → progress → results) — Done

What it does	**Guides the user from URL entry through live scan progress to a full results view: score ring, Pass/Fail badge, severity summary, filterable issue list, and per-issue detail (selector, HTML snippet, WCAG criteria, axe help link).**
Why it exists	Accessibility scanning is slow and technical. Progressive UI states (`queued` → `fetching` → `rendering` → `rule_analysis` → `ai_enrichment` → `scoring` → `completed`) set expectations and match the product goal of a fast, understandable first report.
How it is implemented	**Client:** `web/src/app/ScanExperience.tsx` — submits `POST /api/scans` , polls `GET /api/scans/:id` every ~1s until `completed` or `failed` , renders issue panel and **Export JSON / Export PDF** buttons. **Issue detail UX:** theme-aware panels (light + dark), readable snippet and selector code blocks, severity pills, notes for color-contrast vs landmark rules. **Server:** `web/src/app/page.tsx` passes batch snapshot date via `getBatchSnapshot()` . **Copy:** steps in `how-to-use.ts` , rules in `product-copy.ts` . **QA:** `/fixtures/results` shows layout without a live scan. **Styling:** `globals.css` — orange home CTA, teal accents, Pass/Fail row tints.
How to use it	Open `/` → enter URL or click a **Try example** chip → **Check accessibility** → review results → click issues for detail → **Export JSON** or **Export PDF** when done.

MVP-2. Live Playwright + axe-core analysis — Done

What it does	**Renders the target URL in headless Chrome, waits for DOM readiness, runs axe-core WCAG-tagged rules, and maps violations into Lumen issue objects with severity, selector, and help URLs.**
Why it exists	Many accessibility failures only appear after JavaScript runs (SPAs, lazy content). axe-core is widely used, maps to WCAG success criteria, and keeps detection separate from AI explanation.
How it is implemented	**Orchestration:** `scan-runner.ts` → `runLiveScan()` calls `analyzeUrlWithAxe()` then optional AI, then `scoreFromIssues()` . **Browser:** `live-scan.ts` — singleton Chromium via Playwright (`channel: "chrome"` locally; `PLAYWRIGHT_CHROMIUM=1` in Docker/CI). Navigation retries: `domcontentloaded` → `load` → `commit` (45s timeout each). **axe:** tags `wcag2a` , `wcag2aa` , `wcag21a` , `wcag21aa` , `wcag22aa` ; iframes excluded for stability. **Mapping:** `axe-mapper.ts` → `Issue` type in `types.ts` . **Retry:** one full retry with fresh browser on network errors.
How to use it	Automatic on every live scan. Typical duration: **10–60 seconds** per URL depending on site weight and network.

MVP-3. URL validation + SSRF protections — Done

What it does	**Rejects malformed URLs, non-http(s) schemes, localhost, private IP ranges, link-local addresses, and cloud metadata hostnames before any browser fetch occurs. DNS is resolved and checked again so hostname tricks cannot reach internal networks.**
Why it exists	A public scanner is an SSRF risk vector — without guards, an attacker could probe internal services (`192.168.x.x` , `169.254.x.x` , `metadata.google.internal`).
How it is implemented	**Format:** `validate-url.ts` — trim, add `https://` if missing, require `http:` / `https:` , block `PRIVATE_HOST_PATTERNS` (localhost, 127.x, 10.x, 172.16–31.x, ::1, etc.). Called from `POST /api/scans` before queueing. **DNS SSRF:** `ssrf.ts` — `assertPublicHostname()` resolves hostname and rejects private/reserved IPs; invoked inside `live-scan.ts` before Playwright navigation. **User copy:** blocked-URL explanations in `home-faqs.ts` and `URL_PUBLIC_HINT` in `product-copy.ts` .
How to use it	Paste only public `http` / `https` URLs. Private/local addresses return **400** with a plain-language error — no scan is started.

MVP-4. Per-IP rate limiting — Done

What it does	**Caps how many scan requests (and batch refresh starts) each client IP can make within a time window. Over limit → HTTP 429 with `Retry-After` header and seconds in the JSON body.**
Why it exists	Each live scan launches Chromium and loads a full page — expensive on CPU and RAM. Rate limits prevent abuse and keep the app responsive on shared or low-resource hosts.
How it is implemented	**Core:** `rate-limit.ts` — in-memory fixed-window `Map` keyed by IP (or custom key). **Live scans:** `POST /api/scans` uses `checkRateLimit(ip)` — default **5 requests per 60 seconds** in production (`RATE_LIMIT_MAX` , `RATE_LIMIT_WINDOW_MS`); higher in dev if unset. **Batch refresh:** separate bucket via `getBatchRefreshRateLimit()` in `batch-refresh-config.ts` — **5 per 10 min** when refresh enabled locally; **1 per hour** on production hosts. **IP extraction:** `getClientIp()` reads `x-forwarded-for` / `x-real-ip` behind proxies.
How to use it	Normal use stays under limits. If throttled, wait for the retry period shown in the UI or API response. Self-hosted: tune env vars in `.env.local` .

MVP-5. Persistence (Postgres or PGlite) — Done

What it does	**Stores every scan record and its issues in a relational database so results survive refresh, shareable URLs work, and JSON export reads historical data.**
Why it exists	In-memory storage would lose all scans on restart or deploy. Production needs durable storage; local dev should work with zero database setup.
How it is implemented	**Schema:** `db.ts` — tables `scans` (id, url, status, timestamps, score, severity counts, error) and `issues` (rule, severity, selector, snippet, WCAG criteria JSON, optional AI fields). Index on `issues.scan_id` . **Backends:** PGlite (embedded Postgres) at `web/data/lumen-pg` when `DATABASE_URL` unset; real Postgres `Pool` when set. **Access layer:** async functions in `store.ts` — `saveScan` , `getScan` , `updateScan` , issue upserts on completion. **Health:** `GET /api/health` reports DB mode and connectivity.
How to use it	Local: automatic PGlite — no install. Production: set `DATABASE_URL` (see `web/DEPLOY.md`). Scan URLs like `/` with active result use stored scan IDs internally.

MVP-6. JSON export — Done

What it does	**Downloads a complete machine-readable report after a scan finishes: URL, timestamps, WCAG target level, score, severity counts, and full issue array (including optional AI fields).**
Why it exists	Developers attach reports to tickets, feed QA pipelines, or archive findings.
How it is implemented	**API:** `GET /api/scans/:id/export?format=json` in `export/route.ts`. Returns **409** if scan not `completed`. **Payload fields:** `generator`, `disclaimer`, `scannedUrl`, `scannedAt`, `completedAt`, `wcagLevelTarget`, `score`, `summaryCounts`, `issues[]`. **Download:** `Content-Disposition: attachment; filename="lumen-scan-{id}.json"`. **UI: Export JSON** on results panel in `ScanExperience.tsx`.
How to use it	Complete a scan → click **Export JSON** or call the API with the scan ID.

MVP-6b. Scan report PDF export — Done

What it does	**Generates a formatted PDF report (A4) with disclaimer, URL, score, Pass/Fail, severity totals, and every issue (rule, WCAG, selector, snippet, optional AI text).**
Why it exists	JSON is ideal for tooling; PDF is easier for humans — tutors, stakeholders, portfolio evidence.
How it is implemented	**API:** `GET /api/scans/:id/export?format=pdf`. **Generator:** `scan-report-pdf.ts` — HTML template + Playwright `page.pdf()`. **UI: Export PDF** button beside Export JSON. **Errors:** 409 if scan incomplete; 500 if PDF generation fails.
How to use it	Complete a scan → **Export PDF** on results, or `GET /api/scans/{id}/export?format=pdf`.

MVP-7. Optional AI enrichment — Done

What it does	**After axe finds issues, sends the top N (by severity) to an OpenAI-compatible chat API for a plain-language explanation and suggested fix. Scoring and Pass/Fail are not changed by AI.**
Why it exists	Rule output is accurate but terse. AI helps junior developers understand impact and remediation without replacing axe as the source of truth.
How it is implemented	**Module:** `ai-enrichment.ts` — batches top issues, calls Chat Completions with structured prompt, writes `aiExplanation`, `aiRemediation`, `aiConfidence` onto issue rows. **Pipeline slot:** `scan-runner.ts` sets status `ai_enrichment` before scoring. **Config:** `AI_API_KEY` or `OPENAI_API_KEY`, `AI_BASE_URL`, `AI_MODEL` (default `gpt-4o-mini`), `AI_MAX_ISSUES` (default 5). **Failure handling:** API errors log and skip — scan still completes with axe-only data. **UI flag:** `GET /api/config` → `{ aiTipsEnabled: boolean }` — no secrets exposed to client.
How to use it	Set API key in `.env.local`, restart server. AI sections appear on top issues in results. Without a key, axe results work unchanged.

MVP-8. Anonymous use (no login) — Done

What it does	**Anyone can run scans and view results without creating an account, signing in, or managing sessions. Scans are identified by opaque IDs, not user profiles.**
Why it exists	MVP goal is frictionless "paste URL → get report" for portfolio demos, interviews, and quick checks. Auth adds scope (passwords, GDPR, scan history ownership) beyond v1.
How it is implemented	**No auth middleware** — no NextAuth, cookies, or JWT checks on API routes. **Scan IDs:** `createScanId()` in `store.ts` — random UUID-style identifiers. **Abuse control:** rate limiting by IP instead of per-user quotas. **Data retention:** scans persist in DB/PGlite but are not tied to accounts; no "my scans" UI. **Privacy copy:** `home-faqs.ts` data FAQ explains what is stored locally vs on server.
How to use it	Open the app and scan — no signup. Share results by keeping the browser session or exporting JSON (no account-linked history page yet).

MVP-9. Batch dashboard (28 sites) — Done

What it does	**Shows a pre-computed Pass/Fail dashboard for 28 diverse public websites: summary tiles (tested, passed, failed, issue totals), sortable table (score, severities, date), Pass/Fail badges, and contextual notes for tricky sites.**
Why it exists	Proves Lumen on real-world pages instantly — media SPAs, e-commerce, docs, intentional bad demo (W3C) — without making every visitor wait for 28 live scans.
How it is implemented	**Data:** `website-batch-results.ts` — 28 rows with id, name, url, score, severities, date, optional `note`. **Page:** `web/src/app/batch/page.tsx` — server-rendered summary + table. **Pass/Fail:** `website-pass-fail.ts`. **Guidance:** `website-batch-fail-guidance.ts` / pass guidance for tooltips. **Unified snapshot:** `batch-snapshot-store.ts` — prefers `data/batch-live-snapshot.json` after refresh, else committed TS. **Client-safe types:** `batch-snapshot-types.ts` (no Node `fs` in client bundle). **Read API:** `GET /api/batch/snapshot`.
How to use it	Nav → **Batch results**. Review portfolio benchmark. Use **Run a live scan** to test your own URL. Snapshot date shown in header and home sidebar.

MVP-10. Refresh snapshot (background batch rescan) — Done

What it does	**Rescans all 28 batch URLs live from the `/batch` page, saves updated scores to runtime JSON, and reloads the UI — without blocking other pages or holding one HTTP request open for the full run.**
Why it exists	Committed snapshot dates go stale. Developers and demo hosts need fresh data on demand without running CLI scripts or waiting 30+ minutes on a sequential blocking refresh.
How it is implemented	**Start:** `POST /api/batch/refresh` → **202 Accepted** immediately. **Job state:** `batch-refresh-job.ts` — in-memory `running` / `done` / `failed` with progress `{ current, total, siteName }`. **Rescan engine:** `batch-live-rescan.ts` — `mapWithConcurrency()` (default 2 workers), `withTimeout()` 90s per site, failed sites keep prior row with note. **Poll:** `GET /api/batch/refresh`. **UI:** `BatchRefreshButton.tsx` polls every 2s, shows `Scanning {site} ({N}/28)…`. **Save:** `saveBatchSnapshot()` → `web/data/batch-live-snapshot.json`. **Enable:** dev by default; production requires `BATCH_REFRESH_ENABLED=1`. **Measured performance:** 28/28 in **~88 seconds** on local low-mem setup.
How to use it	`/batch` → **Refresh snapshot**. App stays usable during rescan. Set env vars in `.env.local` for concurrency and rate limits.

MVP-11. Weekly CI batch rescan — Done

What it does	**GitHub Actions automatically live-rescans all 28 sites, validates the snapshot, and commits updated `website-batch-results.ts` to `main` when scores change.**
Why it exists	Runtime JSON from Refresh snapshot does not survive Docker redeploys. Git-committed data keeps production `/batch` accurate after every deploy.
How it is implemented	**Workflow:** `.github/workflows/batch-rescan.yml` — cron `0 4 * * 0` (Sunday 04:00 UTC) + `workflow_dispatch`. **Steps:** checkout → Node 22 → `npm ci` → Playwright Chromium → `npm run batch:sync` (`batch:rescan` → `batch:apply-rescan` → `validate:batch`) → commit/push if diff. **Scripts:** `scripts/batch-rescan.ts`, `apply-batch-rescan.ts`, `validate-batch-snapshot.ts`. **Concurrency group:** `batch-rescan` — no cancel-in-progress. **Timeout:** 120 minutes.
How to use it	Automatic weekly. Manual: GitHub → Actions → **Batch rescan** → Run workflow. Local equivalent: `npm run batch:sync` then git commit.

MVP-12. Responsive layout + viewport CI — Done

What it does	**Adapts layout from 320px phones to 1440px+ desktops: stacked home form on mobile, batch cards on tablet, full tables on laptop/desktop, 44px touch targets, tiered gutters. CI fails if pages overflow horizontally at key widths.**
Why it exists	Accessibility tools must be usable on the devices teams actually carry. Horizontal overflow is a common responsive bug that breaks mobile review workflows.
How it is implemented	**CSS:** `globals.css` breakpoint tokens — `--bp-mobile` 480, `--bp-tablet` 768, `--bp-stack` 800, `--bp-laptop` 1024, `--bp-desktop` 1440. Batch table ↔ card swap, 2×2 summary grids on tablet. **Viewport meta:** `layout.tsx` — zoom allowed (accessibility requirement). **Test script:** `scripts/responsive-viewport-check.ts` — Playwright measures `scrollWidth` vs viewport at 320, 390, 768, 1024, 1280, 1440 on `/`, `/batch`, `/fixtures/results`. **CI:** `responsive-viewport` job in `ci.yml`.
How to use it	Resize browser or DevTools device toolbar. Run `npm run test:responsive` locally (requires `npm run start` in another terminal).

MVP-13. SEO (sitemap, OG, JSON-LD) — Done

What it does	**Exposes search-engine metadata: page titles/descriptions, canonical URLs, Open Graph/Twitter cards, `robots.txt`, dynamic `sitemap.xml`, favicon, and JSON-LD structured data for the public site.**
Why it exists	When deployed, Lumen should be discoverable and preview correctly on LinkedIn/GitHub links — important for portfolio visibility.
How it is implemented	**Metadata:** `seo.ts` + `layout.tsx` — `SITE_TITLE`, `SITE_DESCRIPTION`, keywords, OG image `/og.png`. **URLs:** `site-url.ts` — `getSiteUrl()` from `NEXT_PUBLIC_SITE_URL`. **Routes:** `app/sitemap.ts`, `app/robots.ts`, `app/icon.svg`. **JSON-LD:** WebApplication schema in layout. Production requires `NEXT_PUBLIC_SITE_URL` for correct absolute links.
How to use it	Set `NEXT_PUBLIC_SITE_URL=https://your-domain` on deploy. Submit `https://your-domain/sitemap.xml` in Google Search Console.

MVP-14. Docker + CD workflow — Done

What it does	**Builds a production container with Next.js standalone output, bundled Chromium for Playwright, and pushes to GitHub Container Registry after CI passes on `main`. Optional Render deploy hook triggers hosting.**
Why it exists	Live scanning requires Chrome on the server — Docker standardizes that environment. CD automates repeatable deploys for portfolio/production hosting.
How it is implemented	**Dockerfile:** multi-stage — deps → `npm run build` → runner with `output: standalone`, `PLAYWRIGHT_CHROMIUM=1`, `npx playwright install chromium --with-deps`, port **4376**. **CD:** `.github/workflows/cd.yml` — triggers after CI green on `main`, builds/pushes `ghcr.io/<owner>/lumen-web`. **Secrets:** `RENDER_DEPLOY_HOOK` optional. **Docs:** `web/DEPLOY.md` — env vars, Postgres, SEO URL.
How to use it	`docker build -f web/Dockerfile web`. Or rely on GHCR image from CD pipeline. Configure host with `DATABASE_URL`, `NEXT_PUBLIC_SITE_URL`, optional AI keys.

MVP-15. Deferred: accounts and multi-page crawl — Not yet

What they would do	**Accounts: sign-in, saved scan history, per-user quotas. Multi-page crawl: start at one URL and automatically follow same-domain links to scan many pages in one job.**
Why deferred	MVP delivers **fast single-URL reports**, **JSON + PDF export**, and a **28-site portfolio benchmark**. Accounts add auth, privacy, and retention policy; crawl adds queues, deduplication, timeouts, and bot-wall handling.
What exists today instead	**JSON + PDF export** cover handoff. **Anonymous + rate limit** covers access control lightly. **Single URL + batch list** covers demo and regression without whole-site spidering.
Likely implementation path (future)	Accounts: NextAuth + `user_id` on scans table. Crawl: bounded BFS worker with max depth/pages, same-domain filter, shared scan queue — after public deploy proves demand.

Scoring formula (used across live scans and batch)

Severity	Score penalty
Critical	−25 each
Serious	−15 each
Moderate	−7 each
Minor	−3 each

Formula: start at 100, subtract penalties, clamp 0–100. Zero issues = **100**.

Pass (batch): score ≥ 85 **and** critical = 0.

Labels: Strong (≥85), Fair (60–84), Needs work (<60).

Source: `store.ts` (`WEIGHTS`, `scoreFromIssues`) and `product-copy.ts` (`SCORE_FORMULA`).

API reference (MVP)

Method	Path	Purpose
POST	`/api/scans`	Start scan `{ "url": "https://..." }` → `{ scanId }`
GET	`/api/scans/:id`	Status, score, summary counts
GET	`/api/scans/:id/issues`	Paginated/filtered issue list
GET	`/api/scans/:id/export?format=json`	Download JSON report
GET	`/api/scans/:id/export?format=pdf`	Download PDF report
GET	`/api/batch/snapshot`	Batch date, meta, summary (no rescan)
POST	`/api/batch/refresh`	Start background 28-site rescan (202)
GET	`/api/batch/refresh`	Poll refresh job progress
GET	`/api/config`	`{ aiTipsEnabled }`
GET	`/api/health`	Service + database mode

Software testing approach

Software testing is the process of checking that software behaves as intended, finding defects before users do, and recording evidence that requirements are met. You run planned checks, compare actual results to expected outcomes, and log pass/fail.

Lumen uses testing at **two levels** — the **tool** (does Lumen work?) and the **28 websites** (what do automated axe findings show for each site?).

Level 1 — Testing Lumen (the tool)

Type	What we did	Where
Manual functional testing	Homepage, scan flow, results, export, errors for bad URLs	`TEST_PLAN.md` (UI-*, SEC-*, API-*)
Manual security testing	Block invalid URLs, localhost, private IPs; rate-limit abuse	`TEST_PLAN.md` SEC-01–SEC-04
Automated CI	Lint, typecheck, build, batch validation, responsive viewport	`.github/workflows/ci.yml` — `npm run ci`
Regression testing	Re-run scans and batch sync after code changes	`TEST_PLAN.md` REG-01; `npm run batch:sync`

Purpose: Confirm Lumen can scan, score, persist, export, and protect itself before trusting batch results.

Level 2 — Testing the 28 websites (scan targets)

Type	What we did	Where
Live multi-site scans	Playwright + axe on all 28 public URLs	`npm run batch:sync`, **Refresh snapshot**, weekly CI
Pass/Fail acceptance rule	Pass = score ≥ 85 and critical = 0; Fail otherwise	`website-pass-fail.ts`, `TEST_RESULTS.md`
Results logging	Scores, severities, notes, recommended actions per site	`TEST_RESULTS.md`, `website-batch-results.ts`
Snapshot validation	Totals match rows; Pass/Fail consistent with score formula	`npm run validate:batch`
Scheduled re-test	Weekly rescan; commit if scores change	`.github/workflows/batch-rescan.yml`

Purpose: Build a reproducible portfolio benchmark — diverse real sites (SPAs, media, e-commerce, docs, intentional bad demo) — not a legal WCAG audit.

28-site test flow

1. **Select** 28 public URLs (Google UK, BBC, Netflix, W3Schools, W3C bad demo, etc.).
2. **Scan** each URL — Chrome renders page → axe runs WCAG A/AA tagged rules.
3. **Score** — start at 100; subtract per issue (critical −25, serious −15, moderate −7, minor −3).
4. **Classify** — apply Pass/Fail rule; record in batch table.
5. **Validate** — `validate:batch` checks internal consistency.
6. **Re-run** — manual refresh, CLI sync, or weekly CI to catch site changes.

Latest full refresh: 28/28 OK in ~88 seconds (Aug 2026 background rescan). Example outcome: **8 Pass / 20 Fail** (varies by resync date).

Testing types used (summary)

Testing type	In Lumen
Functional	Scan returns score + issues; batch table renders
Integration	Playwright + axe + DB + API pipeline end-to-end
Regression	Re-scan 28 sites after changes; CI on every push
Security	SSRF blocks, rate limits (TEST_PLAN SEC-*)
Acceptance / MVP	28 sites logged; batch dashboard + export working
Automated	`batch:sync`, `validate:batch`, GitHub Actions
Manual	TEST_PLAN UI checks; reviewer notes on failed sites

Important distinction

Testing the **28 websites** means: *we ran Lumen's automated axe rules on each URL and recorded Pass/Fail by our score rule.* It does **not** mean those sites are legally WCAG compliant or fully accessible — automated rules catch many but not all barriers (manual assistive-technology testing is still required for conformance claims).

Related docs: `TEST_PLAN.md` (tool tests) · `TEST_RESULTS.md` (website Pass/Fail log) · `/batch` (live dashboard)

Quick start

```
cd web
npm install
npm run dev
```

Open **http://localhost:4376**

On memory-limited Windows:

```
npm run build:low-mem
npm run start:low-mem
```

Enable batch refresh locally (`.env.local`):

```
BATCH_REFRESH_ENABLED=1
```

Local development note — plain-text UI incident (Aug 2026)

During local testing, the app briefly appeared as **unstyled plain text** (nav labels run together). The design was **not removed** — CSS failed to load from a stale `.next` build.

Fix: webpack by default, `fix-styles.ps1` , confirm CSS **200** in DevTools → Network. After clean rebuild, UI matched screenshots again.

Disclaimer

Lumen helps teams find and understand accessibility issues faster. It does **not** replace a professional audit, and results should not be treated as legal proof of WCAG conformance.

Generated from project source, README, and screenshots in `docs/screenshots/` .